

Core JS fundamentals

COMPLETE LEARNING GUIDE

What You'll Learn

Every framework, library, and browser API a frontend developer touches is ultimately built on a small core of JavaScript language behavior — variable scoping, type coercion, function semantics, and closures. Getting these fundamentals genuinely solid (not just 'I've seen the syntax before') is what makes debugging fast instead of mysterious, because most bugs in application code trace back to a misunderstanding of one of these core mechanics rather than a framework quirk. This submodule covers variable declarations and scope (let/const vs. the legacy var), how JavaScript's type system and coercion rules actually work, the different ways to define functions and how each handles this, and closures — the mechanism behind private state, callbacks, and much of how hooks work in modern frameworks.

1. let, const, var, and Scope

var is function-scoped and hoisted with its declaration initialized to undefined, which means a var can be 'used' before its declaration line without an immediate error — a frequent source of confusing bugs, especially inside loops. let and const are block-scoped (confined to the nearest {}) and exist in a 'temporal dead zone' from the start of their block until their declaration line, so accessing them early throws a clear ReferenceError instead of silently returning undefined. const prevents reassignment of the variable binding itself, but does not make the value immutable — a const object or array can still have its properties or elements mutated, only the variable can't be pointed at a different value.

- for (var i = 0; ...) inside a loop with an async callback captures the same shared i for every callback; for (let i = 0; ...) creates a fresh binding per iteration.
- const arr = [1,2,3]; arr.push(4) is legal — you're mutating the array's contents, not reassigning the variable.
- Prefer const by default, let only when you know a variable will be reassigned, and avoid var in new code entirely.

JAVASCRIPT

```
// Classic var-in-loop bug
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // logs 3, 3, 3
}

// Fixed with let – each iteration gets its own binding
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0); // logs 0, 1, 2
}
```

2. Types, Coercion, and == vs ===

JavaScript has a small set of primitive types (string, number, boolean, undefined, null, symbol, bigint) plus object (which arrays and functions are special cases of). `typeof` reveals a value's type at runtime, but with two famous quirks: `typeof null` is 'object' (a decades-old bug preserved for compatibility) and `typeof` of an array is also 'object' (use `Array.isArray()` instead). The `==` operator performs type coercion before comparing, producing surprising results like `" == 0` being true, while `===` compares both type and value with no coercion. Modern style guides near-universally mandate `===` specifically to eliminate this class of bug.

- `null == undefined` is true, but `null === undefined` is false — the one case where `==` is sometimes used deliberately.
- `NaN` is the only JS value that is not equal to itself; use `Number.isNaN(x)`, never `x === NaN`.
- Falsy values: `false`, `0`, `-0`, `"`, `null`, `undefined`, `NaN`. Everything else, including `'0'` and `[]` (empty array), is truthy.

3. Functions: Declarations, Expressions, and Arrow Functions

Function declarations (`function greet() {}`) are fully hoisted, so they can be called before their line in the file. Function expressions (`const greet = function() {}`) are not — only the variable declaration is hoisted, not the assignment. Arrow functions (`const greet = () => {}`) are always expressions, have no arguments object, cannot be used as constructors, and — most importantly — do not have their own `this`; they inherit this lexically from the enclosing scope at the time they're defined. This last property is exactly why arrow functions are preferred for callbacks inside class methods or object methods where you want this to keep referring to the outer context.

JAVASCRIPT

```
class Timer {
  seconds = 0;
  start() {
    // Arrow function inherits `this` from start() — refers to the Timer instance
    setInterval(() => { this.seconds++; }, 1000);
  }
}

class BrokenTimer {
  seconds = 0;
  start() {
    // Regular function loses `this` — it's undefined or the global object here
    setInterval(function () { this.seconds++; }, 1000); // bug
  }
}
```

4. Objects and Arrays: The Core Data Structures

Objects are unordered collections of key/value pairs; arrays are ordered, index-based lists that are actually objects under the hood with numeric keys and a special length property. Both are passed and assigned by reference, not by value — assigning an existing object to a new variable copies the reference, not the data, so mutating it through either variable affects the same underlying object. Common array methods split into mutating (push, pop, splice, sort, reverse — modify the original array) and non-mutating (map, filter, slice, concat — return a new array), and knowing which category a method falls into prevents accidental mutation bugs, especially in frameworks like React that rely on detecting new references to trigger re-renders.

- `const a = obj; a.x = 1` also changes `obj.x` — they're the same object in memory.
- `sort()` and `reverse()` mutate the original array in place; `toSorted()`/`toReversed()` (newer, ES2023) return new copies instead.
- `Object.freeze()` makes an object's own properties immutable at the top level only — nested objects inside it remain mutable.

5. Closures: Functions That Remember Their Scope

A closure is a function bundled together with references to its surrounding (lexical) scope, which means the function can continue to access variables from that scope even after the outer function has finished executing. This is the mechanism behind private variables in JavaScript (before classes had real private fields), memoization/caching functions, and — critically for later material — how React's `useState` and custom hooks retain state between renders without any special language feature beyond closures themselves.

JAVASCRIPT

```
function createCounter() {
  let count = 0; // private – only accessible via the closure below
  return {
    increment: () => ++count,
    getValue: () => count,
  };
}

const counter = createCounter();
counter.increment();
counter.increment();
console.log(counter.getValue()); // 2 – `count` persisted between calls
```

6. Control Flow and Truthy/Falsy Checks

JavaScript's if/else, for, while, and switch statements work largely as in other C-family languages, but leaning on truthy/falsy coercion in conditions is idiomatic JS in a way it isn't in stricter languages. `if (array.length)` is a common, correct way to check for a non-empty array, since 0 (empty array's length) is falsy and any positive number is truthy. `switch` uses strict equality (`===`) for its case comparisons, and forgetting a `break` statement causes execution to 'fall through' into the next case — sometimes intentional, but usually a bug.

- `switch` cases fall through without an explicit `break` — group cases deliberately with a comment noting the fallthrough is intentional.
- `if (someValue)` treats 0, "", and NaN as falsy even when they're semantically valid, meaningful values — be explicit (`someValue === 0 ? ...`) when zero is a valid case.

7. Error Handling with try/catch

`try/catch` lets code recover from runtime errors instead of crashing the whole script. Code that might throw goes in the `try` block; the `catch` block receives the thrown error (conventionally named `err` or `e`) and runs recovery logic; an optional `finally` block runs regardless of whether an error occurred, commonly used for cleanup (closing a connection, hiding a loading spinner). Throwing your own errors with `throw new Error('message')` (rather than throwing a plain string) preserves a stack trace, which is essential for debugging in production.

JAVASCRIPT

```
function parseConfig(json) {  
  try {  
    return JSON.parse(json);  
  } catch (err) {  
    console.error('Invalid config JSON:', err.message);  
    return null; // fallback instead of crashing the app  
  } finally {  
    console.log('Config parse attempt finished');  
  }  
}
```

Common Pitfalls

- Using `var` inside a loop with an async callback, expecting each iteration to capture its own value of the loop variable.
- Assuming `const` makes an object or array immutable — it only prevents reassigning the variable, not mutating its contents.
- Using `==` instead of `===` and getting bitten by unexpected coercion (`"" == 0`, `null == undefined`).
- Passing a regular function as a callback inside a class method and being surprised that `this` is no longer the instance.`
- Checking for NaN with `x === NaN`, which is always false — NaN is never equal to itself.
- Mutating an array with a mutating method (`sort`, `splice`) when the original, unmodified array was still needed elsewhere.
- Throwing a plain string (`throw 'error'`) instead of an `Error` object, losing the stack trace needed to debug it.

Key Takeaways

- Prefer `const` by default, `let` when reassignment is needed, and avoid `var` entirely in new code.
- `===` avoids an entire category of coercion bugs that `==` silently permits.
- Arrow functions inherit `this` lexically — that's precisely why they're the right choice for callbacks inside class/object methods.`
- Objects and arrays are reference types — copying the variable does not copy the underlying data.
- Closures are the mechanism, not a framework feature — they're what lets a returned function 'remember' variables from its creating scope.

Learn More

MDN — JavaScript Guide

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>

javascript.info — The Modern JavaScript Tutorial

<https://javascript.info/>

MDN — Closures

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures>